

Multiple Load Balancers per Service



can we have multiple load balancer used with one service

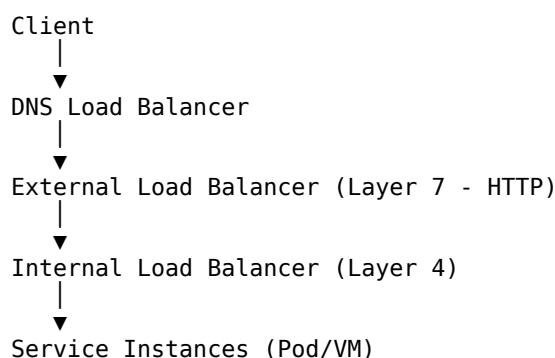
Yes, **a single service can use multiple load balancers**. This is actually very common in modern distributed systems and microservice architectures. The idea is usually to balance traffic at **different layers or for different purposes**.

Let's look at the main scenarios.

1 Multiple Load Balancers in Different Layers (Most Common)

A service may sit behind **two levels of load balancing**.

Example architecture:



Example technologies:

- DNS load balancing → Amazon Route 53
- External load balancer → NGINX / HAProxy / AWS Application Load Balancer
- Internal load balancing → Kubernetes Service

So **one service is still behind multiple load balancers**, but each one has a different role.

2 Different Load Balancers for Different Traffic Types

Sometimes the same service uses **separate load balancers for different protocols**.

Example:

```

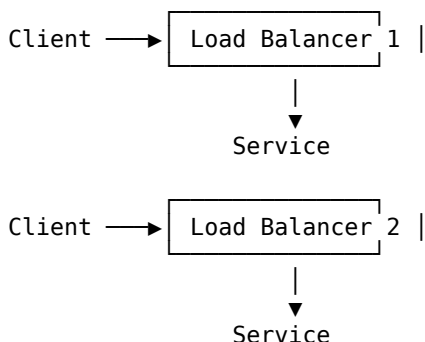
Internet Users → HTTP Load Balancer → Service
Mobile API → API Gateway / Load Balancer → Service
Internal Services → Internal Load Balancer → Service
  
```

Example tools:

- API traffic → Kong API Gateway
- Internal service traffic → Envoy Proxy

3 Active-Active Load Balancers (High Availability)

Multiple load balancers can run **in parallel** to avoid a single point of failure.



DNS or Anycast routing decides which LB the client reaches.

Example:

- Cloudflare Load Balancing
- AWS Elastic Load Balancing

4 Example in Microservices (like Netflix)

Companies like Netflix use **multiple layers of load balancing**:

1. Edge load balancer (internet traffic)
2. API gateway
3. Service-to-service load balancing (client-side)

Netflix often uses **client-side load balancing** with libraries like Netflix Ribbon.

✓ Interview answer (short version):

Yes, a service can be behind multiple load balancers. This is common for layered architectures where DNS, external load balancers, and internal load balancers distribute traffic for scalability, fault tolerance, and different traffic types.

💡 If you want, I can also explain a **very common system design interview question**:

Why companies like Netflix moved from Load Balancer-based routing to Client-Side Load Balancing.

It's a really important microservices concept.



when one service makes calls to another service through feign client then why service discovery is required we can call the load balancer of the other service that were managing the multiple instances of that service

Good question. This is exactly how microservices evolved.

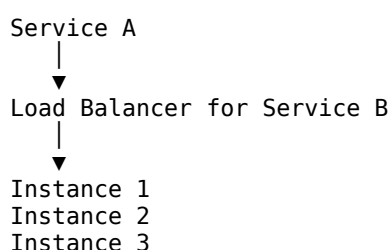
Short answer: **You *can* call another service through its load balancer, but service discovery removes the need for a dedicated load balancer for every service and makes the system more dynamic.**

Let's break it down.

1 Your Idea: Call the Load Balancer of the Service

Suppose **Service A** wants to call **Service B**.

Architecture you are thinking about:



Example call:

`http://service-b-lb/api/orders`

Yes, this works. Many early microservice systems used this approach.

Example tools used for this:

- NGINX
- HAProxy
- AWS Elastic Load Balancing

But this creates **several problems**.

2 Problem 1: Too Many Load Balancers

Imagine you have **100 microservices**.

You would need:

100 services
100 load balancers

Infrastructure becomes expensive and complex.

Companies like Netflix realized this problem early in their microservices journey.

3 Problem 2: Instances Change Dynamically

In microservices:

- instances scale up
- instances die
- new containers start

Example:

Service B instances

10.1.2.5
10.1.2.8
10.1.2.12

Tomorrow:

10.1.3.4
10.1.3.6
10.1.3.7
10.1.3.9

Updating load balancers constantly becomes difficult.

4 Service Discovery Solution

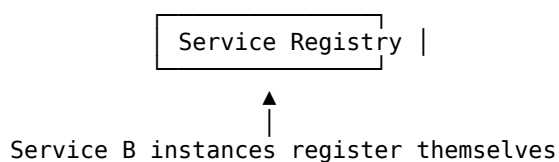
Instead of calling a load balancer, services ask a **registry**:

"Where are the instances of Service B?"

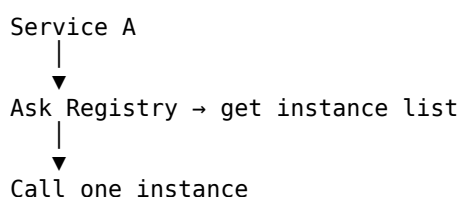
Example registry:

- Netflix Eureka
- Consul
- Apache ZooKeeper

Architecture:



Then:



5 What Feign + Service Discovery Does

When using Feign:

Java

```
@FeignClient("order-service")
```

Feign does:

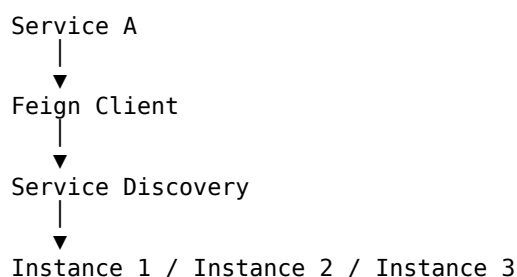
- 1 Ask registry (like Netflix Eureka)
- 2 Get all instances of **order-service**
- 3 Choose one instance
- 4 Send request

This is called **client-side load balancing**.

Spring uses:

- Spring Cloud LoadBalancer
(previously Netflix Ribbon)

Architecture:

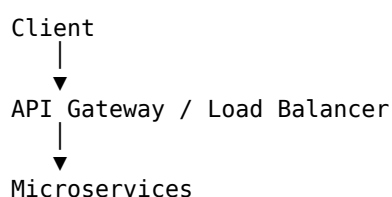


No dedicated load balancer needed.

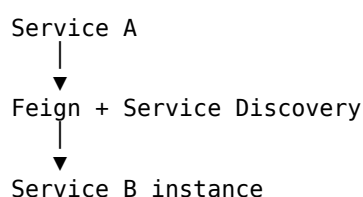
6 Real Microservice Architecture

Usually companies use **both**.

External traffic:



Internal calls:



Example API gateway:

- Spring Cloud Gateway
 - Kong
-

Interview Answer (Perfect)

You can say:

Yes, services can call another service through a load balancer, but in microservices this becomes inefficient because every service would need its own load balancer. Service discovery solves this by maintaining a dynamic registry of service instances. Feign clients query the registry and use client-side load balancing to select an instance, removing the need for a dedicated load balancer for every service.

 If you want, I can also explain **one very tricky interview concept**:

Why Netflix removed server-side load balancing and moved to client-side load balancing.

This is one of the **most asked microservices interview questions**.